

Step 1: Check GCC

```
gcc --version
```

```
g++ --version
```

If not installed:

```
sudo apt update
```

```
sudo apt install build-essential
```

Step 2: Save your code

```
nano bfs_dfs.cpp
```

```
CTRL + O → Enter → CTRL + X
```

Step 3: Compile with OpenMP

```
g++ -fopenmp bfs_dfs.cpp -o hpc
```

Step 4: Run

```
./hpc
```

code :

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
#include <omp.h>
```

```
#include <chrono>
```

```
#include <random>
```

```
using namespace std;
```

```
// ————— BUBBLE SORT —————
```

```

void bubbleSort_Sequential(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j+1])
                swap(arr[j], arr[j+1]);
}

```

// Parallel Bubble (Odd-Even Sort)

```

void bubbleSort_Parallel(vector<int>& arr) {
    int n = arr.size();

    for (int phase = 0; phase < n; phase++) {

        if (phase % 2 == 0) {
            #pragma omp parallel for
            for (int i = 0; i < n - 1; i += 2) {
                if (arr[i] > arr[i+1])
                    swap(arr[i], arr[i+1]);
            }
        } else {
            #pragma omp parallel for
            for (int i = 1; i < n - 1; i += 2) {
                if (arr[i] > arr[i+1])
                    swap(arr[i], arr[i+1]);
            }
        }
    }
}

```

```
// _____ MERGE SORT _____
```

```
void merge(vector<int>& arr, int l, int m, int r) {
```

```
    vector<int> left(arr.begin()+l, arr.begin()+m+1);
```

```
    vector<int> right(arr.begin()+m+1, arr.begin()+r+1);
```

```
    int i = 0, j = 0, k = l;
```

```
    while (i < left.size() && j < right.size())
```

```
        arr[k++] = (left[i] <= right[j]) ? left[i++] : right[j++];
```

```
    while (i < left.size()) arr[k++] = left[i++];
```

```
    while (j < right.size()) arr[k++] = right[j++];
```

```
}
```

```
void mergeSort_Sequential(vector<int>& arr, int l, int r) {
```

```
    if (l >= r) return;
```

```
    int m = (l + r) / 2;
```

```
    mergeSort_Sequential(arr, l, m);
```

```
    mergeSort_Sequential(arr, m+1, r);
```

```
    merge(arr, l, m, r);
```

```
}
```

```
// Better Parallel Merge using TASKS (important fix)
```

```
void mergeSort_Parallel(vector<int>& arr, int l, int r, int depth = 0) {
```

```
    if (l >= r) return;
```

```

int m = (l + r) / 2;

if (depth < 3) {
    #pragma omp task shared(arr)
    mergeSort_Parallel(arr, l, m, depth+1);

    #pragma omp task shared(arr)
    mergeSort_Parallel(arr, m+1, r, depth+1);

    #pragma omp taskwait
} else {
    mergeSort_Sequential(arr, l, m);
    mergeSort_Sequential(arr, m+1, r);
}

merge(arr, l, m, r);
}

//————— UTILITY —————

vector<int> generateRandom(int n) {
    vector<int> v(n);
    mt19937 rng(42);
    uniform_int_distribution<int> dist(1, 100000);

    for (auto &x : v) x = dist(rng);

    return v;
}

```

```

bool isSorted(const vector<int>& v) {
    for (int i = 0; i < v.size()-1; i++)
        if (v[i] > v[i+1]) return false;
    return true;
}

// ----- MAIN -----

int main() {

    const int BUBBLE_N = 5000;
    const int MERGE_N = 200000;

    cout << "=====\n";
    cout << " Parallel Sorting using OpenMP\n";
    cout << "=====\n";

    // Bubble Sort
    auto orig = generateRandom(BUBBLE_N);

    auto arr1 = orig;
    auto t1 = chrono::high_resolution_clock::now();
    bubbleSort_Sequential(arr1);
    auto t2 = chrono::high_resolution_clock::now();

    cout << "\nBubble Sequential: " << (isSorted(arr1) ? "YES" : "NO");
    cout << " | Time: "
        << chrono::duration<double, milli>(t2-t1).count() << " ms\n";

    auto arr2 = orig;
    t1 = chrono::high_resolution_clock::now();

```

```

bubbleSort_Parallel(arr2);

t2 = chrono::high_resolution_clock::now();


cout << "Bubble Parallel: " << (isSorted(arr2) ? "YES" : "NO");
cout << " | Time: "
    << chrono::duration<double, milli>(t2-t1).count() << " ms\n";


// Merge Sort
auto big = generateRandom(MERGE_N);

auto arr3 = big;
t1 = chrono::high_resolution_clock::now();
mergeSort_Sequential(arr3, 0, arr3.size()-1);
t2 = chrono::high_resolution_clock::now();

cout << "\nMerge Sequential: " << (isSorted(arr3) ? "YES" : "NO");
cout << " | Time: "
    << chrono::duration<double, milli>(t2-t1).count() << " ms\n";

auto arr4 = big;

t1 = chrono::high_resolution_clock::now();

#pragma omp parallel
{
    #pragma omp single
    mergeSort_Parallel(arr4, 0, arr4.size()-1);
}

t2 = chrono::high_resolution_clock::now();

```

```
cout << "Merge Parallel: " << (isSorted(arr4) ? "YES" : "NO");  
cout << " | Time: "  
    << chrono::duration<double, milli>(t2-t1).count() << " ms\n";  
  
return 0;  
}
```

◆ Step 1: Install WSL

Open PowerShell (Admin):

```
wsl --install
```

Restart PC

◆ Step 2: Open Ubuntu terminal

Search → "Ubuntu"

◆ Step 3: Install GCC + OpenMP

```
sudo apt update
```

```
sudo apt install build-essential
```

◆ Step 4: Run your HPC code

```
nano program.cpp
```

```
g++ -fopenmp program.cpp -o run
```

```
./run
```